

FUSIONCORE v_1

Master Roadmap — Physics-Aware Temporal RUL Estimation for C-MAPSS

Predictive In-orbital Network — PiNet

Author: Michele E. J. Maestrini

1. Project Directive

1.1 Introduction

FusionCore v_1 introduces PiNet — the Predictive in-orbital Network — a compact physics-guided temporal neural architecture for Remaining Useful Life (RUL) estimation and operational risk-band classification of commercial turbofan engines. PiNet consumes the validated 91-feature FD00u feature manifold produced by FusionCore v_0 Phase 3 unchanged and pairs a long-memory Temporal Convolutional Network (TCN)¹ with a small physics token branch carrying the gas-path thermodynamic interpretation of degradation. The two branches share a fused embedding from which two output heads decode scalar RUL and a three-class operational risk band: Healthy, Warning, and Critical.

The v_1 programme is scoped around three principles: simplification, comparability, and falsifiability. PiNet is built as a two-branch deterministic backbone. The headline evaluation is performed on the NASA C-MAPSS official test set, ensuring direct comparability with the published PHM benchmark literature. The Siamese metric learning with Shannon regularisation [1]) is preserved as a single pre-declared advanced ablation executed on a representative sample subset, compute-bounded for Apple Silicon hardware, with rollout to full FD00u contingent on positive findings.

FusionCore v_2 is sketched at the end of this document as a probabilistic bolt-on to the v_1 PiNet backbone — conformal RUL prediction intervals² and calibrated risk-band probabilities — but v_2 scope is not presupposed by v_1 design decisions.

1.2 Scope and Objectives

FusionCore v_1 is scoped to the NASA C-MAPSS dataset exclusively (subsets FD001–FD004). The N-CMAPSS dataset is out of scope. The v_1 programme delivers the following measurable outcomes, in execution order:

(1) Consume the v_0 Phase 3 91-feature parquet output unchanged. (2) Construct PiNet as a two-branch architecture: a long-memory TCN over 30-cycle windows and a small physics-token MLP over a curated sub-vector of the same 91 features. (3) Train PiNet on the Internal Train partition under a two-term loss combining the NASA asymmetric RUL score and a class-weighted cross-entropy term for the three-class risk band. (4) Validate convergence on the held-out Internal Validation partition. (5) Evaluate the trained PiNet on the NASA C-MAPSS official test set, reporting RMSE, NASA asymmetric score, MAE, and Critical-band precision, recall, and F_2 , both per subset and pooled. (6) Carry forward the v_0 Phase 4 XGBoost results as the head-to-head comparator without re-evaluation. (7) Execute one pre-declared

¹ Temporal Convolutional Networks are explained in detail in Appendix A.1.

² Conformal prediction is explained in detail in Appendix A.2.

advanced ablation on a representative sample subset adding a triplet metric-learning regulariser and a Shannon entropy term to the Stage A loss. (8) Serialise PiNet weights, hyperparameters, and trajectory-level prediction artefacts to enable a probabilistic v_2 extension.

1.3 Declared Limitations

Two items registered as mandatory in FusionCore v_0 Phase 3 are addressed within v_1 scope. The Yeo–Johnson power transformation on the high-skewness virtual sensors CPR and $E_{thermal}$ is applied to the neural-network input matrix only, not to the XGBoost input matrix.

The rationale is straightforward and is grounded in standard statistical learning theory. Tree-based models such as XGBoost are invariant to any monotone transformation of an input feature, because tree splits depend only on the rank ordering of feature values, not on their numerical magnitude [2]. Consequently, applying Yeo–Johnson to CPR would not change a single XGBoost prediction. By contrast, neural networks are highly sensitive to input distributional shape because gradient-descent learning is sensitive to feature scale and skew; an input feature with skewness 41.20 (as CPR exhibits in the v_0 Phase 3 output) will produce gradient updates dominated by extreme tail values during training, regardless of the architecture downstream. The principled solution is to maintain two parallel input matrices: the untransformed 91-feature matrix consumed by XGBoost (preserving direct comparability against the v_0 Phase 4 results), and a Yeo–Johnson-transformed variant of the same 91 features consumed by PiNet. The transformation is fitted on Internal Train data only, frozen, and applied read-only to Internal Validation and the NASA test set.

The third item from the v_0 Phase 3 mandatory list — the SHAP ablation of s_{16} (farB) against the dead-sensor noise floor — is scheduled as a Phase v1.5 diagnostic deliverable.

1.4 Relationship to FusionCore v_0 and Forward Versions

FusionCore v_0 completed five phases producing a validated 91-feature physics-aware feature manifold³, an XGBoost baseline with SHAP forensic audit (RMSE 14.85 cycles, NASA Asymmetric Score 4,336.3, Critical-band $F_2 = 0.9339$ on the NASA official test set), and three SOTA neural comparators (DeepAR, TFT, PatchTST). FusionCore v_1 inherits the v_0 feature manifold and the v_0 Phase 4 XGBoost evaluation results as-is. The XGBoost model is not re-trained or re-evaluated in v_1 scope; its v_0 Phase 4 numbers on the NASA official test set carry forward and serve as the comparator for PiNet on the same test set. The v_0 SOTA neural comparators are not re-evaluated in v_1 either.

FusionCore v_2 (probabilistic extension) and v_3 (full-deployment system on N-CMAPSS) are bookmarked in the project constitution but are not presupposed by v_1 design decisions. The PiNet backbone is serialised in TorchScript form for reproducibility. Use as a pretrained-encoder transfer artefact for v_2 or v_3 is a forward-decision out of scope for this document.

³ The 91-feature manifold structure is documented in the FusionCore v_0 Phase 3 Technical Analysis. The features comprise raw Z-scored sensor channels, kinematic derivatives, virtual thermodynamic sensors, cumulative fatigue indices, and fault-family flags.

2. Dataset Reference

2.1 NASA C-MAPSS — Subset Characteristics

The NASA C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) benchmark [3] is the de facto standard testbed for turbofan RUL estimation in the Prognostics and Health Management literature. It comprises four run-to-failure subsets, each containing a separate training corpus (engines run to failure) and test corpus (engines truncated mid-life with the true RUL of the truncation point provided as the ground-truth label). Each engine trajectory consists of 21 sensor measurements, 3 operational settings, and a unit identifier, with RUL in flight cycles as the regression target.

Table 1 — C-MAPSS training corpus statistics (run-to-failure trajectories).

Subset	Engines	Rows	Max Cycle	Fault Modes	Regimes
FD001	100	20,631	362	HPC degradation only	1 (sea-level ISA)
FD002	260	53,759	378	HPC degradation only	6 (full envelope)
FD003	100	24,720	525	HPC + fan compound	1 (sea-level ISA)
FD004	249	61,249	543	HPC + fan compound	6 (full envelope)
Total	709	160,359	543	—	—

Table 2 — C-MAPSS test corpus statistics (truncated trajectories with, held-out RUL labels).

Subset	Test Engines	Test Rows	Role in v_1
FD001	100	13,096	Headline evaluation, single-fault single-regime
FD002	259	33,991	Headline evaluation, single-fault multi-regime
FD003	100	16,596	Headline evaluation, dual-fault single-regime
FD004	248	41,214	Headline evaluation, dual-fault multi-regime
Total	707	104,897	Pooled FD00u headline evaluation

The piecewise-linear capped RUL target follows Heimes [4] with a plateau cap $R_{cap} = 125$ cycles. The cap is grounded physically in the Paris' Law crack-initiation phase [5] — the period during which fatigue-driven defect propagation is below the detection threshold of onboard sensors — and operationally as a uniform RUL ceiling that prevents the loss surface from being dominated by extremely long-lived engines.

2.2 Dataset Input to v_1

FusionCore v_1 receives the 91-feature FD00u parquet output of FusionCore v_0 Phase 3 as its fixed dataset input. All preprocessing artefacts — Frozen Regime Dictionary, per-regime z-score parameters, Cox PH $\hat{\beta}$ coefficients, Shewhart upper warning and control limits, inverse-frequency regime weights, and family-specific UWL thresholds — are embedded in the parquet artefact set. No preprocessing step is repeated in v_1 , with the single exception of the Yeo–Johnson power transformation on CPR and $E_{thermal}$ for the neural input matrix only, fitted on Internal Train data and frozen as a v_1 artefact.

The v_1 programme operates with a three-tier partition discipline. The training corpus (709 engines pooled across FD001–FD004 as FD00u) is split 80:20 by engine boundary using `GroupShuffleSplit`

on the composite key (`subset_origin, unit_id`)⁴ into Internal Train and Internal Validation. The NASA official test corpus (707 engines, 104,897 rows) is the headline evaluation partition.

Table 3 — v_1 partition discipline.

Partition	Engines	Rows	Role in v_1	When touched
Internal Train	567 (~80%)	129,331	Loss optimisation; weight updates	Every batch of every epoch
Internal Validation	142 (~20%)	31,028	Convergence monitoring; epoch selection	Once per epoch
NASA Official Test	707	104,897	Headline evaluation; literature comparability	Exactly once at programme close

The Iron Wall Protocol applies to the NASA test set as the headline evaluation partition: preprocessing artefacts are read-only, no fitting, no sequential tuning, and the partition is touched exactly once at programme close. This protocol ensures direct comparability with every published RMSE and NASA score in the C-MAPSS literature, since the entire field reports against the NASA official test set.

2.3 Regime and Class Imbalance

FusionCore v_0 Phase 3 confirmed two structural imbalances in the FD00u corpus that persist in v_1 . The high-altitude cruise regime is approximately twice as frequent as the next most common regime in FD002 and FD004; uncorrected, this would cause gradient-based models to optimise preferentially on cruise observations. The inverse-frequency regime weight w_r , computed once on Internal Train and frozen, corrects this at the loss level for regime r with n_r rows among N total rows across R regimes:

$$w_r = \frac{N}{R \times n_r} \quad [\text{Eq. 1}]$$

The second imbalance is structural under-representation of the terminal phase ($RUL < 30$ cycles). A 300-cycle engine at window stride 1 contributes roughly 270 windows above $RUL = 30$ against only 30 windows below — a 9:1 imbalance in the most safety-critical region. The mitigations are described in §7.

3. PiNet Architecture

3.1 Architectural Philosophy

PiNet is a two-branch hybrid: a long-memory Temporal Convolutional Network for trajectory structure, and a small physics token branch for thermodynamic grounding. The two branches share a fused embedding from which two supervision heads decode the prognostic outputs. The two-branch design is grounded in the observation that the short-range degradation patterns⁵ which a convolutional pathway would learn — sudden step changes in a sensor reading, brief volatility bursts, drift onsets — are already encoded in the kinematic features (rolling means and standard deviations) of the v_0 feature manifold;

⁴ `GroupShuffleSplit` is the `scikit-learn` partition strategy that ensures all rows belonging to the same group (here, the same engine) are placed together in the same partition. This prevents within-engine row leakage from training to validation.

⁵ In machine-learning terminology, a "motif" is a short, repeating sub-pattern in a time series — for example, a particular shape of sensor spike that occurs repeatedly during a specific failure mode. See Appendix §A.7 for a worked example.

representational responsibility is therefore divided between long-memory temporal structure (TCN) and absolute physical state (physics tokens).

The physics branch is retained — and this is the design choice most likely to draw a question from a non-specialist reviewer, so the rationale deserves direct treatment.

3.1.1 Why a Physics Branch When the Inputs Already Contain the Physics?

A reasonable objection runs as follows. The 91-feature FusionCore v_0 manifold already contains gas-path thermodynamic variables: pressure ratios, isentropic efficiencies, EGT margins, cumulative fatigue indices, and so on. If the TCN consumes all 91 features, is the physics branch not duplicating information the TCN can already, see? The answer is no, for three connected reasons grounded in representation learning theory and gas-turbine engineering practice.

1. Joint feature presence is not the same as architectural separation. The full 91-feature input matrix mixes raw z-scored sensor channels, kinematic derivatives (rolling means and standard deviations), virtual thermodynamic sensors (CPR , η_c , $E_{thermal}$), cumulative fatigue indices, and one-hot fault-family flags. When the TCN convolves over this mixed matrix, the eight or so physically interpretable scalars per cycle compete with the other ~ 83 channels for representational capacity inside the TCN's receptive field. There is no architectural guarantee that the network preserves the physical interpretation of those scalars in its hidden activations — empirically, deep networks tend to encode whatever combination of inputs minimises training loss, which is rarely the same as the engineer's mental model. The physics branch is a small, dedicated pathway that processes only the physically interpretable scalars through a shallow MLP, producing an embedding contribution whose meaning is auditable.
2. Physical features have a different role from kinematic features. A rolling standard deviation of a sensor reading captures local volatility — useful for short-horizon anomaly detection but agnostic to whether the sensor is approaching a thermodynamic limit. A virtual sensor like the compressor pressure ratio $CPR = s_7/s_5 = P_{30}/P_2$ is a physical quantity with a known certified envelope; its absolute level (not just its volatility) is operationally meaningful. The physics branch preserves this distinction by giving the absolute physical scalars their own forward-graph location.
3. The physics branch is computationally cheap. It takes a sub-vector of approximately 11 scalars per cycle (sensors, virtual sensors, cumulative fatigue indices, the Cox proportional-hazards score, and the inverse-frequency regime weight), passes them through a two-layer fully connected MLP ($\approx 2,500$ parameters total), and concatenates the output to the TCN embedding before fusion. The added compute is well under one percent of the TCN cost. The interpretability gain is large; the cost is negligible.

In short: the 91-feature input to the TCN provides predictive signal across all degradation modes; the physics branch ensures that gas-path thermodynamic structure is explicitly preserved in the embedding space rather than being implicitly buried inside whatever representation the TCN happens to learn.

3.2 Input Representation

The TCN branch input is a sliding window of the (Yeo–Johnson-transformed) 91-feature matrix produced by FusionCore v_0 Phase 3, with dimensions $W \times 91$ where $W = 30$ cycles is the window length. The 30-cycle choice follows the C-MAPSS literature precedent of Heimes [4] and Li et al. [6] and is supported by the v_0 Phase 1 autocorrelation analysis which identified a temporal memory of approximately lag 17 cycles. A 30-cycle window comfortably covers the autocorrelation tail with margin.

Window stride is 1 cycle, with engine boundaries enforced by the composite key (`subset_origin`, `unit_id`) so that windows never cross from one engine to another. For trajectories shorter than 30 cycles (rare in C-MAPSS — the minimum trajectory length is 128 cycles in FD001 and longer elsewhere — but defensible engineering practice nonetheless), forward-fill padding replicates the first observed cycle leftward to fill the window. This is physically defensible because the Frozen Regime Dictionary

assigns $z = 0$ as the within-regime healthy baseline, making forward-fill a reasonable approximation of early healthy operation. An attention mask flags padded positions for the TCN.

3.3 Branch 1 — Long-Memory Temporal Convolutional Network

The TCN branch [7] applies a stack of dilated causal one-dimensional convolutions to the windowed feature trajectory. Causal means each output position depends only on current and past inputs, never future ones — a property of architectural importance in any prognostic context where look-ahead access would constitute a leakage failure. Dilation means the convolution skips over inputs at exponentially increasing intervals, allowing a small kernel to cover a long temporal range without exploding the parameter count.

For kernel size $K = 3$ and L dilated layers with dilation factors $1, 2, 4, \dots, 2^{L-1}$, the effective receptive field of the stack covers the following number of cycles:

$$\text{ERF} = 1 + (K - 1)(2^L - 1) \quad [\text{Eq. 2}]$$

For $L = 5$ this evaluates to $\text{ERF} = 1 + 2 \times 31 = 63$ cycles, which exceeds the $W = 30$ window length and ensures every output position depends on all input positions in the window. Recommended configuration values for the primary v_1 TCN are summarised in Table 4. These are fixed at the start of the primary experiment and are not tuned during training; deviation from these values requires a pre-declared ablation entry.

Table 4 — PiNet TCN branch configuration (primary v_1 values).

Hyperparameter	Value	Source / Rationale
TCN blocks L	5	$\text{ERF} = 63 \text{ cycles} > W = 30$
Kernel size K	3	Bai et al. [7] standard
Dilations	$\{1, 2, 4, 8, 16\}$	Doubling pattern, standard TCN [7]
Hidden channels	64	Li et al. [6] on C-MAPSS
Dropout	0.15	Regularisation on a 16 GB MPS budget
Normalisation	LayerNorm	Batch-size-agnostic; standard TCN practice
Residual connections	Yes	Bai et al. [7]
Causality	Mandatory	No look-ahead in any layer

3.4 Branch 2 — Physics Token Branch

The physics branch consumes a curated sub-vector of the 91-feature input — specifically, the physically interpretable scalars enumerated in Table 5 — and processes them through a two-layer fully connected MLP with ReLU activations and batch normalisation. The output is concatenated with the TCN branch output before fusion. The shallow architecture is intentional: the physics tokens are already semantically meaningful and do not require deep nonlinear transformation.

Table 5 — Physics branch token set.

Token	Source Feature	Physical Meaning	v_0 Phase 3 Origin
z_{s4}	Normalised EGT (T_{50})	HPC efficiency loss indicator	Active sensor; Gate 2
z_{s7}	Normalised P_{30}	Fault-family discriminator	Active sensor; Gate 2
z_{s9}	Normalised N_c	Multi-regime degradation indicator	Active sensor; Gate 2
CPR	$s_7/s_5 = P_{30}/P_2$	Compressor pressure ratio	Virtual sensor; Gate 5
η_c	Isentropic efficiency	Departure from ideal compression	Virtual sensor; Gate 5
CFI_{s4}	Cumulative Fatigue Index, EGT ⁶	Cumulative thermal fatigue proxy	Gate 6 monotonicity
CFI_{s9}	Cumulative Fatigue Index, N_c	Cumulative mechanical fatigue proxy	Gate 6 monotonicity
CFI_{s7}	Cumulative Fatigue Index, P_{30}	Cumulative pressure cycle fatigue	Gate 6 monotonicity
Hazard score	$\exp(\hat{\beta}^T X)$ from Cox PH ⁷	Conditional failure probability ratio	v_0 Phase 3 G12 frozen
w_r	Inverse-frequency regime weight	Regime imbalance correction	v_0 Phase 3 G4 resolved
Fault family flag	One-hot single/dual-fault	Fault mode context	Gate 8

3.5 Fusion and Output Heads

The TCN branch produces a temporal pooled representation of the windowed trajectory. Following the recalibration recommendation from the prior review, this representation is the concatenation of the TCN's last-timestep hidden state (which answers "given everything observed up to now, what is the current state?") and the mean-pooled hidden state across the window (which captures degradation evidence distributed across the window). The physics branch produces a separate token embedding. The two are concatenated and passed through a two-layer fusion MLP with layer normalisation to produce a shared embedding e of dimension $d_e = 128$ following Li et al. [6].

Two output heads decode the embedding. The RUL regression head is a two-layer MLP mapping e to a scalar RUL estimate clipped to $[0, R_{cap} = 125]$. The risk-band classification head maps e to a three-class softmax over Healthy ($RUL \geq 80$), Warning ($30 \leq RUL < 80$), and Critical ($RUL < 30$). The 30-cycle Critical boundary is calibrated to minimum actionable maintenance lead time per Nowlan and Heap's Reliability-Centred Maintenance analysis [8] and to the v_0 Phase 2 Gate 3 evidence; the 80-cycle Warning boundary is the upper threshold below which proactive maintenance logistics planning is justified by the cumulative fatigue trend observed in the v_0 Gate 6 evidence.

⁶ The Cumulative Fatigue Index family is explained in detail in Appendix A.5.

⁷ The Cox Proportional Hazards model is explained in detail in Appendix A.6.

3.6 Architecture Diagram

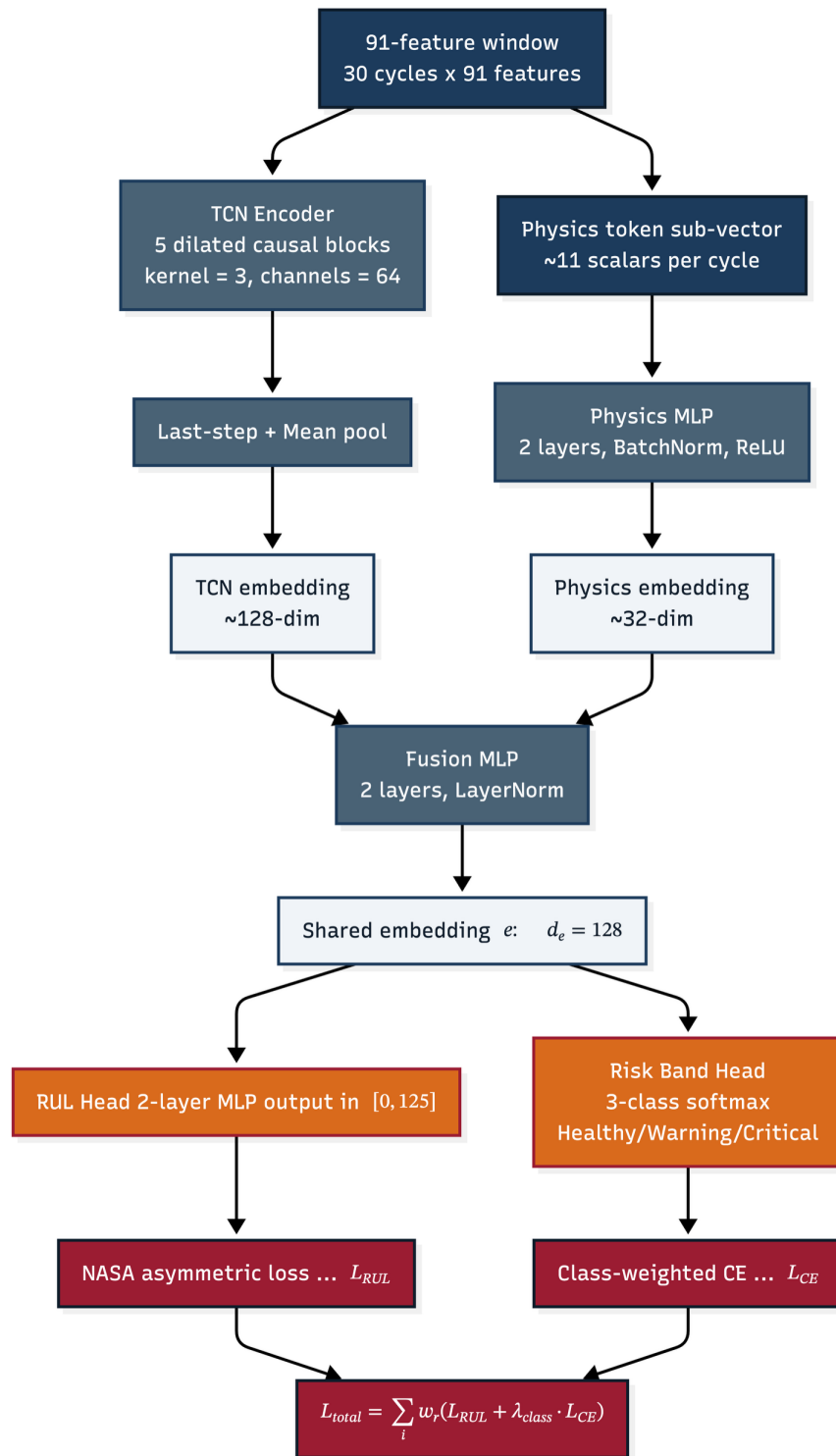


Figure 1: PiNet architectural pipeline

4. Training Regime

4.1 Stage A — Backbone Pretraining

Stage A trains PiNet end-to-end on the 80% Internal Train partition under a two-term loss. The training procedure is deliberately simple: a single optimisation passes through the data with one composite loss, no metric-learning sampling, no auxiliary heads, no embedding-collapse diagnostics. This is the falsifiability-first design — if the primary PiNet underperforms XGBoost, the cause must be diagnosable from a single set of training curves.

4.2 The Two-Term Loss Function

The primary v_1 loss combines a regression term and a classification term, both observation-weighted by the inverse-frequency regime weight w_r from §2.3:

$$L_{total} = \sum_i w_{r(i)} (L_{RUL,i} + \lambda_{class} \cdot L_{CE,i}) \quad [\text{Eq. 3}]$$

The first term $L_{RUL,i}$ is the NASA asymmetric score on observation i , defined precisely below. The second term $L_{CE,i}$ is the class-weighted cross-entropy for the three-class risk band on observation i . The mixing coefficient $\lambda_{class} = 0.3$ is fixed at the start of the primary experiment, justified against the 13.2% terminal-fraction class imbalance and the operational priority of Critical-class discrimination. Any deviation from $\lambda_{class} = 0.3$ is treated as a pre-declared ablation experiment and reported separately.

The reasoning behind $\lambda_{class} = 0.3$ is the following.⁸ The terminal-fraction class imbalance is 13.2% — meaning 13.2% of training rows fall into the Critical band ($RUL < 30$ cycles), while approximately 87% fall into Healthy or Warning. If the two loss terms L_{RUL} and L_{CE} were given equal weight ($\lambda_{class} = 1.0$), the classification term would dominate the gradient signal whenever a Critical example appeared, because class-weighted cross-entropy on a heavily under-represented class produces large per-example loss values. The model would then learn primarily to discriminate Critical from non-Critical at the expense of the regression objective. Conversely, if λ_{class} were too small (say 0.1), the classification head would receive insufficient gradient signal to learn the three-band decision boundary at all, and the risk-band output would collapse toward the majority class. The value $\lambda_{class} = 0.3$ sits at the geometric midpoint of the operationally defensible range $[0.1, 1.0]$ on a log scale, providing a balanced gradient contribution: roughly one part classification signal to three parts regression signals. A worked numerical example is given in Appendix A.7.

The NASA asymmetric score [3] penalises late predictions (underestimated RUL) more heavily than early ones, reflecting the asymmetric operational cost in which a missed warning is more dangerous than a premature one:

$$s(e) = \exp\left(-\frac{e}{13}\right) - 1 \text{ for } e < 0 (\text{early}) \quad [\text{Eq. 4a}]$$

$$s(e) = \exp\left(\frac{e}{10}\right) - 1 \text{ for } e \geq 0 (\text{late}) \quad [\text{Eq. 4b}]$$

⁸ See Appendix §A.8 for a worked numerical example showing how the gradient contribution of each loss term changes with λ_{class} , and why 0.3 produces a balanced training signal.

where $e = \hat{y} - y$ is the prediction error in cycles. The asymmetry parameters $\{13,10\}$ were calibrated by NASA against operational fleet maintenance economics; the late-penalty exponent $1/10$ is steeper than the early-penalty exponent $1/13$, encoding the engineering judgement that an unanticipated in-service failure exceeds the cost of premature removal by a factor that grows exponentially with the magnitude of the underestimate.

The class-weighted cross-entropy L_{CE} uses inverse class-frequency weights computed once on Internal Train and frozen. This addresses the 9:1 structural imbalance in the terminal class (§2.3) by up-weighting Critical-class examples in the gradient signal, complementing the asymmetry already present in L_{RUL} .⁹

4.3 Fixed Hyperparameter Strategy

FusionCore v_1 adopts fixed hyperparameter values sourced from peer-reviewed PHM literature [4, 6, 7] and the v_0 Phase 4 ablation evidence. A Bayesian Hyperparameter Optimisation is deferred to a potential v_2 work-package. The decision rests on three pillars: (i) the 80:20 Internal partition concentrates all labelled trajectories in Train and Validation, leaving no independent fine-tune partition for HPO without violating the Iron Wall Protocol; (ii) fixed values provide a deterministic baseline against which ablation evidence is unambiguously interpretable; and (iii) the XGBoost comparator in the v_0 Phase 4 evaluation used its published default configuration, so PiNet must be evaluated under an equivalently fixed configuration for fair comparison.

Table 7 — Fixed hyperparameter values for FusionCore v_1 Stage A.

Hyperparameter	Fixed Value	Source / Rationale
Embedding dim d_e	128	Li et al. [6] optimal trade-off on C-MAPSS
TCN layers L	5	ERF = 63 cycles > W
TCN kernel K	3	Bai et al. [7]
TCN hidden channels	64	Li et al. [6]
Window length W	30 cycles	Heimes [4]; v_0 Phase 1 ACF lag 17 + margin
Physics MLP	2 layers, BatchNorm, ReLU	Shallow intentional
Batch size	64	Apple Silicon MPS 16 GB headroom
Learning rate	1×10^{-3}	Adam default; Li et al. [6]
Optimiser	AdamW	Standard PHM practice
Epochs	80 with early stopping patience 15	Empirical on v_0 SOTA
λ_{class}	0.3	Calibrated to 13.2% terminal class imbalance

Hyperparameter immutability for v_1 — Once Stage A training commences, the values in Table 7 are frozen for the duration of the v_1 programme. Any deviation must be logged as an explicit ablation experiment and reported separately from the headline benchmark. Sequential informal tuning against the held-out NASA test set would constitute a Test-set-leakage event under the Iron Wall Protocol and is prohibited.

⁹ See Appendix §A.9 for a worked numerical example illustrating how inverse-frequency class weights up-weight under-represented Critical examples in the gradient signal.

4.4 Pre-Declared Advanced Ablation — Triplet Metric Learning with Shannon Regularisation

A single pre-declared advanced ablation is included in the v_1 programme to provide methodological infrastructure for the FusionCore v_2 probabilistic extension. The ablation augments the primary two-term loss with a semi-hard triplet metric-learning regulariser and a Shannon entropy term¹⁰:

$$L_{abl} = L_{total} + \lambda_{trip} \cdot L_{triplet} + \lambda_{shan} \cdot L_{shan} \quad [\text{Eq. 5}]$$

with $\lambda_{trip} = \lambda_{shan} = 0.5$ as fixed initialisation values. The triplet term enforces the geometric constraint that, in the embedding space, anchor windows are closer to same-health-stage positive windows than to different-health-stage negative windows by at least a margin $\alpha = 0.5$ following Schroff et al. [9]. The Shannon term penalises entropy depletion in the stage-probability distribution, preventing the "embedding-collapse failure mode of pure metric learning [10] and providing the methodological link to the MSc-dissertation work [1].

4.5.1 Compute-Bounded Execution Strategy

The advanced ablation is materially more compute-intensive than the primary experiment. Triplet sampling adds a per-batch computational overhead in the order of 30–50% over the primary loss, and the additional optimisation surface typically requires more epochs to converge. On a 16 GB Apple Silicon MPS budget the full FD00u corpus would be a multi-day training task per ablation seed, which is not realistic given the available hardware.

The execution strategy is therefore tiered. The advanced ablation is first run on a representative sample subset of FD00u — a stratified slice preserving the joint distribution of subset (FD001–FD004), regime, and fault family. The sample subset is constructed by sampling engines with a target size of approximately 30% of FD00u Internal Train (≈ 170 engines, $\approx 39,000$ rows). On this subset the ablation runs in 4–6 hours per seed on the M4. If the ablation produces a positive finding — defined as a tighter held-out RUL distribution, an improved Critical-class F_2 , or improved separation of same-stage versus different-stage embedding distances at the $p < 0.05$ level on a Kolmogorov–Smirnov test — the rollout to the full FD00u Internal Train is authorised as a single overnight run. If the sample-subset finding is null or negative, the rollout is not executed, and the advanced ablation is reported as a null result alongside the primary headline. This is a defensible compute-management strategy that does not bias the headline experiment.

5. Evaluation Framework

5.1 Headline Evaluation Partition — NASA Official Test Set

The headline evaluation partition for FusionCore v_1 is the NASA C-MAPSS official test set: 707 truncated engine trajectories totalling 104,897 rows across the four subsets. PiNet is evaluated on this partition exactly once at programme close. The evaluation is non-iterative; no hyperparameter, no

¹⁰ A triplet (anchor, positive, negative) is "semi-hard" when the negative is further from the anchor than the positive is, but only by less than the margin α . Triplets that are too easy (negative already very far) provide no gradient signal; triplets that are too hard (negative closer than positive) can destabilise training. Semi-hard mining selects only the informative middle band. See Appendix §A.3 (extended) and §A.10 for the full explanation with a worked example.

¹¹ "Entropy depletion" means that the model becomes excessively confident, producing SoftMax outputs that approach $(1,0,0)$ or $(0,0,1)$ instead of well-calibrated distributions like $(0.7,0.25,0.05)$. This is a known failure mode of pure metric learning where the encoder shrinks each class to a single point in the embedding space, destroying useful within-class variation. The Shannon term keeps the distribution distributed. See Appendix §A.11.

architectural choice, no training-time decision may depend on the test-set evaluation result. This is the central tenet of the Iron Wall Protocol applied at the test-set tier.

Two headline metrics are reported, consistent with the entire C-MAPSS PHM benchmark literature [3, 4, 6]: the root-mean-square error (RMSE) and the NASA asymmetric score S . Both are computed over the per-engine final-cycle predictions (one prediction per test engine, against the held-out true RUL provided by NASA) per subset and pooled across FD00u:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2} \quad [\text{Eq. 6}]$$

$$S = \sum_{i=1}^N s(\hat{y}_i - y_i) \quad [\text{Eq. 7}]$$

Supplementary metrics include MAE, Critical-class precision, recall, and the F_2 -score per the v_0 four-quadrant evaluation framework. The F_2 -score weights recall twice as heavily as precision and is therefore the appropriate single-number metric for the safety-critical Critical class, in which a missed detection (false negative) carries operational cost asymmetrically larger than a false alarm.

5.2 Benchmark Comparator — XGBoost (v_0 Phase 4 Carry-Forward)

The sole head-to-head comparator in v_1 is XGBoost [11], which was trained and evaluated by FusionCore v_0 Phase 4 on the same NASA official test set. The v_0 Phase 4 XGBoost results are carried forward unchanged: RMSE 14.85 cycles, NASA Asymmetric Score 4,336.3, Critical-band $F_2 = 0.9339$. XGBoost is not re-trained or re-evaluated in v_1 . This is correct and defensible: tree-based models are deterministic given a fixed training corpus and fixed hyperparameters, and the v_0 Phase 4 numbers are reproducible artefacts on the same partition discipline used in v_1 .

The v_0 Phase 4 SOTA neural comparators (DeepAR, TFT, PatchTST) are not carried forward as v_1 comparators because their v_0 Phase 4 NASA Asymmetric Scores were several thousand-fold worse than XGBoost (the worst was approximately 2,320× worse), placing them outside the operational viability band. Reporting them alongside PiNet would obscure rather than inform the headline comparison.

5.3 Critical-Recall Safety Gate

PiNet performance against XGBoost is graded against a three-tier safety gate:

Table 8 — Critical-recall safety gate for v_1 headline evaluation.

Outcome	Condition	Programme decision
Strong Pass	PiNet Critical recall $\geq$ XGBoost Critical recall	PiNet declared the operationally superior model
Pass with Advisory	PiNet Critical recall $\geq$ XGBoost Critical recall -0.02	PiNet declared comparable; flagged as a watch item for v_2
Fail	PiNet Critical recall $<$ XGBoost Critical recall -0.02	PiNet not adopted as a replacement; failure analysis in Phase v1.6

A model that improves RMSE but materially worsens Critical recall is not considered operationally superior. NASA score is given priority over RMSE when the two metrics disagree, unless residual analysis demonstrates that the NASA score degradation is caused by benign early prediction (over-conservatism) rather than late prediction (under-prediction of risk).

6. Phase-by-Phase Execution Plan

FusionCore v_1 is structured across seven execution phases. Each phase has defined inputs, key outputs, and acceptance gates. No phase may commence before the preceding phase's gates have passed. All gates pair a programmatic numeric assertion with the relevant supporting visual: visuals are mandatory deliverables, and every visual is backed by a numeric assertion that can be verified in code.

Table 9 — FusionCore v_1 execution plan (seven phases).

Phase	Description	Key Outputs	Acceptance Gates
Ph- v_1 .0	Load and verify v_0 Phase 3 parquet output; confirm partition dimensions; assert composite-key disjointness; fit Yeo–Johnson on Train	Verified X , y matrix shape and integrity for the NN path (Yeo–Johnson-transformed); confirmation that the XGBoost-path matrix is identical to the v_0 Phase 4 input (untransformed); YJ artefact (transformer fitted on Internal Train only)	Row counts match Table 3; engine disjointness verified; halts on mismatch
Ph- v_1 .1	Window construction for TCN ($W = 30$) with forward-fill; composite-key enforcement; ¹² class-weighted CE weights; risk-band labels	PyTorch Dataset and DataLoader producing ($B \times W \times 91$) batches	Window shape verified; no engine-boundary crossing; no NaN
Ph- v_1 .2	PiNet implementation: TCN, physics MLP, fusion MLP, two output heads.	PiNet PyTorch module; architecture + pipeline (Fig. 1)	Forward pass dimensional consistency; TorchScript export; parameter count
Ph- v_1 .3	Stage A primary training: two-term loss; inverse-frequency regime weighting; fixed hyperparameters per Table 7; checkpoint every epoch	Trained PiNet weights; per-epoch loss curves; TorchScript export	Loss convergence; no NaN gradients; Critical recall non-trivial mid-training
Ph- v_1 .4	NASA test-set headline evaluation: PiNet vs XGBoost (v_0 carry-forward) on RMSE, NASA S , MAE, Critical precision/recall/ F_2 , per subset and pooled	Headline comparison table; per-subset breakdown; figure set for final report	Test set touched exactly once; safety gate (Table 8) results recorded
Ph- v_1 .5	Diagnostics and the s_{16} SHAP audit: residual analysis by risk band; per-engine trajectory overlays; SHAP-against-noise-floor check	Diagnostic report; SHAP attribution table	S_{16} contribution decision recorded; PiNet-S contingency trigger evaluated

¹² Forward-fill is a padding technique used when an engine's trajectory is shorter than the window length $W = 30$. The first observed cycle is replicated leftward to fill the missing positions before cycle 0. An attention mask flags the replicated positions so the TCN ignores them in gradient computation. See Appendix §A.12.

Phase	Description	Key Outputs	Acceptance Gates
Ph- $v_{1.6}$	Advanced ablation: triplet + Shannon on representative sample subset; conditional rollout to full Internal Train	Ablation comparison; KS-test embedding evidence; rollout decision	Sample-subset ablation completes; rollout if positive finding; null result reported either way

7. Sampling and Class Imbalance

Class and regime imbalance are addressed at three levels: input-feature weighting (the inverse-frequency regime weight from §2.3), loss-function weighting (class-weighted cross-entropy from §4.2), and batch composition.

7.1 Batch Composition

A representative-but-stabilised batch composition is recommended for the primary v_1 training, drawing on standard practice in imbalanced-classification literature [12]:

Table 10 — Recommended batch composition (training only; validation and test distributions are preserved natural).

Risk band	RUL range	Natural fraction in Internal Train	Target batch share
Healthy	$RUL \geq 80$	$\approx 60\%$	40–50%
Warning	$30 \leq RUL < 80$	$\approx 27\%$	25–30%
Critical	$RUL < 30$	$\approx 13.2\%$	25–30%

In plain language, a "training batch" is the small set of examples (here, 64 windows) that the optimiser uses to compute one gradient update step. By default, the sampler draws batches uniformly at random from the training corpus, which would produce batches with the natural class proportions of the corpus: roughly 60% Healthy, 27% Warning, 13% Critical. Under this natural distribution, a typical 64-window batch contains only about 8 Critical examples, which is too few for the gradient signal on the Critical class to be reliable in any single update step. The model effectively learns the Critical decision boundary very slowly, only as Critical examples accumulate across many batches. The recommended batch composition in Table 10 modifies the sampler to over-sample Critical (and to a lesser extent Warning) so that every batch contains 16–19 Critical examples — enough for the per-batch Critical-class gradient signal to be statistically stable. This is a standard imbalanced-classification mitigation drawn from He and Garcia [12], applied at sampler level only — neither the validation nor the test partition distributions are resampled, so all reported metrics remain on the natural class distribution.

The over-representation of Critical and under-representation of Healthy at batch level is a training stabilisation mechanism only; the Internal Validation and NASA test set distributions are never resampled, and all reported metrics are computed against natural distributions.

8. Novel Contribution Register

The contributions registered for FusionCore v_1 against the current state of the published PHM and machine-learning literature are scoped conservatively to the level of evidential support available. Claims are stated explicitly in terms of the published literature they extend or differ from, with citation references in Section 13.

Table 11 — FusionCore v_1 novel contribution register.

Contribution	Adjacent literature	FusionCore v_1 position
Two-branch physics-token TCN over a validated 91-feature gas-path manifold for C-MAPSS RUL estimation	TCNs applied to C-MAPSS [6, 13]; physics-informed neural networks more generally [14] but not in this two-branch token-MLP form on C-MAPSS	Original synthesis of TCN with a dedicated physics-token MLP branch on the validated v_0 feature manifold
Triplet metric learning with Shannon entropy regularisation as an embedding-shaping mechanism for turbofan trajectory windows (advanced ablation)	Triplet loss applied to image classification [9] and earlier work on plant disease classification [1]; not previously applied to PHM time-series embeddings on C-MAPSS	Original transfer of the dissertation methodology [1] from binary image classification to a multi-class time-series PHM context
Forward-coupled $v_1 \rightarrow v_2$ design enabling conformal RUL prediction intervals over the v_1 embedding without backbone retraining	Conformal prediction [15] is not yet established practice in the C-MAPSS PHM benchmark literature	Original positioning of the v_1 deterministic backbone as the substrate for v_2 probabilistic calibration

9. Forward Look — FusionCore v_2

FusionCore v_2 is a probabilistic bolt-on to a successful PiNet v_1 backbone. The v_2 scope is intentionally narrow: it adds calibrated uncertainty quantification on top of the v_1 deterministic outputs, without retraining the backbone and without expanding into deployment infrastructure. The v_2 architecture is described here in outline only; full v_2 specification is a separate document.

9.1 The Conformal Prediction Layer

Conformal prediction [15] is a distribution-free statistical framework for converting any point-prediction model into a prediction-interval model with provable finite-sample coverage guarantees. The technique is well-suited to PiNet v_1 because it operates entirely as a post-hoc calibration step on a held-out partition, requires no architectural changes to the backbone, and produces intervals with mathematically grounded coverage properties.

The basic mechanism is the following. Given a trained PiNet model and a held-out calibration partition (a subset of Internal Validation, or a separate calibration partition cut from Internal Train at v_2 scope), the absolute residuals $|r_i| = |\hat{y}_i - y_i|$ are computed for each calibration example. The empirical $(1 - \alpha)$ -th quantile of these residuals, $\hat{q}_{1-\alpha}$, is the conformal radius. For any new test example with point prediction $\hat{y}_*$, the conformal prediction interval at miscoverage level α is:

$$C_\alpha(\hat{y}_*) = [\hat{y}_* - \hat{q}_{1-\alpha}, \hat{y}_* + \hat{q}_{1-\alpha}] \quad [\text{Eq. 8}]$$

Under the assumption of exchangeability between calibration and test data — which holds for the C-MAPSS test set, since the test engines are drawn from the same generative simulator as the training engines — the interval $C_\alpha(\hat{y}_*)$ contains the true y_* with probability at least $1 - \alpha$. For a 90% coverage interval ($\alpha = 0.10$), this gives a calibrated band around each PiNet RUL estimate that respects the empirical residual distribution of the trained model.

In plain language, the goal of conformal prediction is to wrap a single number (the model's point estimate) with an interval that expresses calibrated honesty about how wrong the model can be. A useful analogy is the weather forecast: a forecaster who says "tomorrow will be 22°C" is making a point prediction, while a forecaster who says "tomorrow will be 22°C, with a 90% chance of being between 18°C and 26°C" is making a calibrated interval prediction. The conformal layer is the second kind. PiNet alone produces the "22°C" — the conformal wrapper produces the "18–26°C with 90% confidence." For

maintenance planning, the second is far more actionable: a maintenance manager can budget logistics for the worst plausible RUL within the interval rather than for the single point estimate alone.

A worked example: if PiNet predicts $\hat{y}_* = 44$ cycles and the calibrated 90% conformal radius is 13 cycles, the reported interval is $[31, 57]$ cycles. The maintenance interpretation is direct: "the engine is most likely to require maintenance in 31 to 57 cycles, with 90% calibrated confidence."

9.2 Risk-Band Probability Calibration

PiNet v_1 's risk-band classification head produces softmax probabilities, but uncalibrated softmax outputs from neural networks are systematically over-confident [16]. The v_2 extension applies temperature scaling — a one-parameter calibration model trained on Internal Validation — to convert raw softmax outputs into reliability-calibrated probabilities. Calibrated probabilities allow operationally meaningful statements such as "among engines flagged with $P(\text{Critical}) \approx 0.25$, approximately one in four should actually transition to Critical within the relevant horizon," which is the language of statistical decision support rather than the language of unanchored model confidence.

9.3 Out-of-Scope for v_2

The following are explicitly out of v_2 scope: deployment dashboard infrastructure, fleet-level ranking interfaces, real-time inference pipelines, persistent prediction stores, and alert state machines. These are operational deployment concerns and are bookmarked for v_3 or later. The v_2 deliverable is a calibrated probabilistic extension of v_1 , not a deployment system.

10. Leakage Risk Register

The leakage risks relevant to v_1 are listed in Table 12, ordered sequentially. The register is scoped to risks live under the v_1 design; risks tied to capabilities outside v_1 scope (Bayesian HPO, HDF5 streaming, distributed-training gradient sync, Spark partition shuffles) are instated only when the corresponding capability is introduced.

Table 12 — FusionCore v_1 live leakage risk register.

ID	Description	Category	Mitigation
LR-01	Engine appearing in multiple partitions	Hard leakage	Two-way GroupShuffleSplit by (subset_origin, unit_id) at 80:20 inherited from v_0 Phase 2; the NASA test set is held disjoint by NASA construction; programmatic non-intersection assertion at Ph- $v_1.0$
LR-02	Statistical parameters (z-score, YJ) fitted on Validation or test	Hard leakage	All preprocessing artefacts inherited read-only from v_0 Phase 3 (Frozen Regime Dictionary, z-score parameters, Cox PH $\hat{\beta}$, etc.). The new v_1 Yeo–Johnson transformer for CPR and $E_{thermal}$ is fitted on Internal Train only, then applied read-only to Internal Validation and the NASA test set — identical leakage discipline to the v_0 z-score parameters. The transformer is serialised as a frozen v_1 artefact at the end of Phase Ph- $v_1.0$ and is not re-fitted at any subsequent point in the programme.

ID	Description	Category	Mitigation
LR-03	Cox PH coefficients contaminated by future data	Hard leakage	$\hat{\beta}$ adopted from v_0 Phase 3 G12, frozen; read-only application to Validation and test partitions
LR-04	Rolling-window statistics crossing engine boundaries	No leakage	Within-engine computation enforced by composite-key grouping in <code>DataLoader</code>
LR-05	Cumulative fatigue indices crossing engine boundaries	No leakage	Within-engine accumulation from cycle 0 of each trajectory
LR-06	Windowing across engine boundaries in Stage A	Hard leakage if unchecked	Composite-key enforcement in the windowing function; per-batch assertion
LR-07	Internal Validation accessed during training/tuning	Hard leakage	Validation used for early stopping only; fixed hyperparameters eliminate tuning leakage
LR-08	NASA test set accessed before programme close	Hard leakage	Test set touched exactly once at Ph- v_1 .4; access logged; no iteration permitted post-evaluation

11. Bibliography

- [1] M. E. J. Maestrini, "Does climate change have an effect on agricultural plant pathogens? A Few-Shot Learning approach for binary plant disease classification on IoT platforms," MSc Dissertation, 2024.
- [2] L. Breiman, J. H. Friedman, R. A. Olshen and C. J. Stone, Classification and Regression Trees, Wadsworth, Belmont, CA, 1984.
- [3] A. Saxena and K. Goebel, "Turbofan Engine Degradation Simulation Data Set," NASA Ames Prognostics Data Repository, Moffett Field, CA, 2008.
- [4] F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in Proceedings of the International Conference on Prognostics and Health Management (PHM 2008), Denver, CO, 2008.
- [5] P. C. Paris and F. Erdogan, "A critical analysis of crack propagation laws," Journal of Basic Engineering, vol. 85, no. 4, pp. 528–533, 1963.
- [6] X. Li, Q. Ding and J.-Q. Sun, "Remaining useful life estimation in prognostics using deep convolution neural networks," Reliability Engineering & System Safety, vol. 172, pp. 1–11, 2018.
- [7] S. Bai, J. Z. Kolter and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modelling," arXiv preprint arXiv:1803.01271, 2018.
- [8] F. S. Nowlan and H. F. Heap, "Reliability-Centred Maintenance," Report No. AD-A066-579, United Airlines / U.S. Department of Defense, 1978.
- [9] F. Schroff, D. Kalenichenko and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 815–823, 2015.
- [10] K. Q. Weinberger and L. K. Saul, "Distance metric learning for large margin nearest neighbour classification," Journal of Machine Learning Research, vol. 10, pp. 207–244, 2009.
- [11] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.
- [12] H. He and E. A. Garcia, "Learning from Imbalanced Data," IEEE Transactions on Knowledge and Data Engineering, vol. 21, no. 9, pp. 1263–1284, 2009.
- [13] Y. Wang, Y. Zhao and S. Addepalli, "Remaining Useful Life Prediction using Deep Learning Approaches: A Review," Procedia Manufacturing, vol. 49, pp. 81–88, 2020.
- [14] M. Raissi, P. Perdikaris and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," Journal of Computational Physics, vol. 378, pp. 686–707, 2019.
- [15] V. Vovk, A. Gammerman and G. Shafer, Algorithmic Learning in a Random World, Springer, New York, 2005.
- [16] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in Proceedings of the 34th International Conference on Machine Learning (ICML), vol. 70, pp. 1321–1330, 2017.

- [17]** D. R. Cox, "Regression models and life-tables," Journal of the Royal Statistical Society, Series B, vol. 34, no. 2, pp. 187–220, 1972.
- [18]** M. A. Miner, "Cumulative damage in fatigue," Journal of Applied Mechanics, vol. 12, no. 3, pp. A159–A164, 1945.
- [19]** L. van der Maaten and G. Hinton, "Visualising Data using t-SNE," Journal of Machine Learning Research, vol. 9, pp. 2579–2605, 2008.

Appendix A — Technical Concept Expansions

This appendix expands the technical concepts referenced via footnotes in the main body of the roadmap. Each section is a standalone explanation suitable for a non-specialist reader.

A.1 Temporal Convolutional Networks

A Temporal Convolutional Network is a stack of one-dimensional convolutional layers configured to respect the causal direction of time. In a standard convolution, each output position is computed from a small window of input positions on either side. In a causal convolution, the window extends only backward in time: each output at time t is computed from inputs at times $t, t - 1, t - 2, \dots$ but never from $t + 1$ onward. This causality is achieved structurally by left-padding the input by one less than the kernel size and then truncating from the right.

Dilation is the second key ingredient. A dilated convolution skips over input positions at exponentially increasing intervals. With dilation factor d , the convolution touches positions $t, t - d, t - 2d, \dots$. Stacking L layers with dilations $\{1, 2, 4, \dots, 2^{L-1}\}$ produces an effective receptive field that grows exponentially with depth while keeping the parameter count linear in L . This is what allows a TCN to model very long temporal dependencies efficiently — Bai et al. [7] show that TCNs match or exceed LSTM and Transformer performance on a wide range of sequence-modelling benchmarks while training faster and being easier to debug.

In the FusionCore v_1 context, the TCN consumes a 30-cycle window of the 91-feature manifold and produces a hidden representation at each cycle. The final-cycle hidden state and the mean of all cycle hidden states are concatenated and forwarded to the fusion layer.

A small, worked example clarifies how dilation produces an exponentially growing receptive field. Consider a trajectory of 31 cycles indexed 0 through 30 (representing one 30-cycle window of EGT z-scores from a single engine). The TCN stack uses 5 layers with dilations $\{1, 2, 4, 8, 16\}$ and kernel size $K = 3$. At layer 1 (dilation 1), the convolution at output cycle 30 looks at input cycles 28, 29, 30 — three immediately adjacent positions. At layer 2 (dilation 2), the convolution at output cycle 30 looks at output positions 26, 28, 30 of layer 1 — skipping every other position. At layer 3 (dilation 4) it looks at positions 22, 26, 30. By layer 5 (dilation 16) it looks at positions $30 - 32 = -2, 14, 30$ — covering 32 cycles back from the current position. Because the input is only 30 cycles long, the layer-5 convolution's leftmost tap falls off the front of the window and is handled by the same forward-fill padding strategy described for short trajectories. The total effective receptive field per Eq. 2 is $1 + (3 - 1)(2^5 - 1) = 1 + 62 = 63$ cycles, more than twice the 30-cycle window length, so by the final layer every output position has had access to every input position in the window.

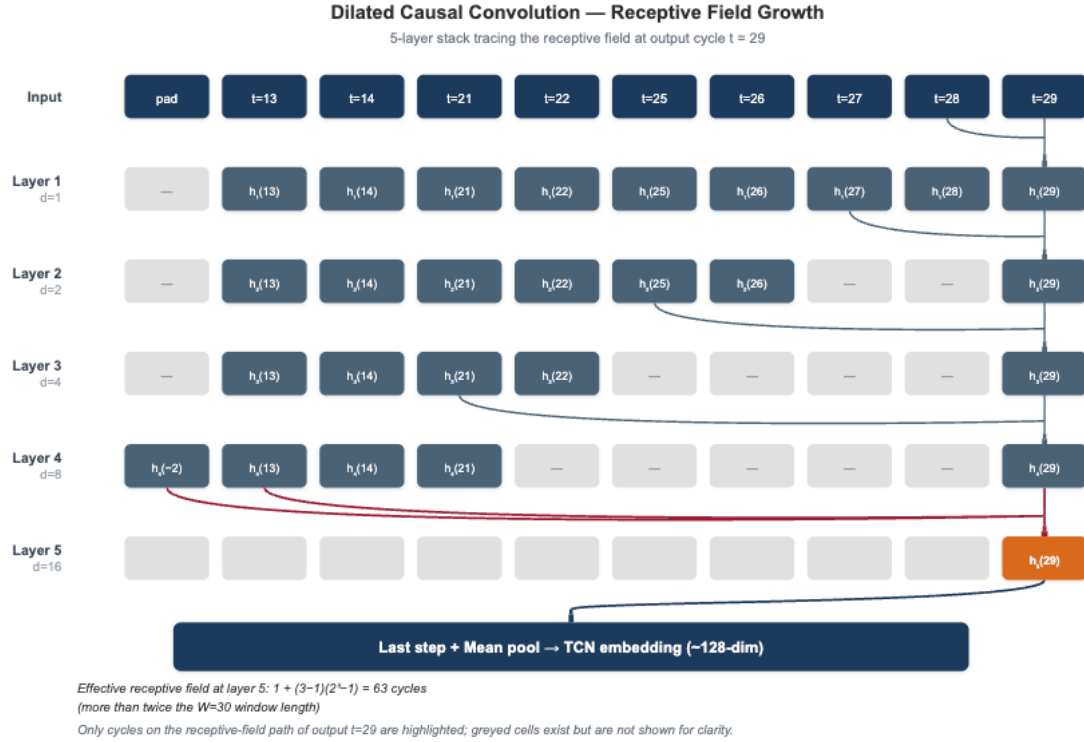


Figure 2: Dilated causal Temporal Convolutional Network: 5-layer stack tracing the receptive field at output cycle $t = 29$, with effective receptive field 63 cycles spanning the full 30-cycle window.

A.2 Conformal Prediction Intervals

In plain language: conformal prediction takes any model that produces a single number (a point prediction) and converts it into a model that produces a number plus an honest interval around it. The honesty comes from a mathematical theorem that says, if the calibration data and the future data look broadly similar in distribution, the true value will fall inside the interval at least 90% of the time when we ask for a 90% interval — independent of how good or bad the underlying model is. A poorly trained model will produce wide intervals (large radius); a well-trained model will produce narrow intervals (small radius). Either way, the calibration honesty is preserved. This is why conformal prediction is preferred over SoftMax confidence scores or Bayesian neural-network output spreads: those methods are model-dependent and often miscalibrated, while conformal intervals come with a finite-sample coverage guarantee that holds for any base model.

Conformal prediction [15] is a post-hoc statistical wrapper that converts any point-prediction model into a prediction-interval model with provable finite-sample coverage. The core idea is straightforward and worth seeing as a small, worked example.

Suppose PiNet is trained, and it produces predictions $\hat{y}_i$ on a held-out calibration partition of size n for which the true RUL labels y_i are known. Compute the absolute residuals $|r_i| = |\hat{y}_i - y_i|$ for each calibration example, sort them in ascending order, and select the $[(1 - \alpha)(n + 1)]$ -th value from the sorted list. Call this value $\hat{q}_{1-\alpha}$. For a new test example with prediction $\hat{y}_*$, the conformal interval at miscoverage level α is simply $[\hat{y}_* - \hat{q}_{1-\alpha}, \hat{y}_* + \hat{q}_{1-\alpha}]$.

The mathematical guarantee is that, under the (mild) assumption that calibration and test data are exchangeable, the true y_* lies inside the interval with probability at least $1 - \alpha$. A worked numerical example: if 100 calibration residuals have a 90th-percentile value of 13 cycles, then for any new PiNet prediction $\hat{y}_* = 44$, the 90% conformal interval is $[31, 57]$ cycles. Among 1,000 such test predictions, at least 900 should contain the true RUL.

For the FusionCore v_2 extension, conformal prediction is attractive because it requires no architectural change to PiNet, no retraining, and no distributional assumption beyond exchangeability. The cost is that it consumes a calibration partition — typically a fraction of Internal Validation reserved specifically for this purpose at v_2 scope.

A.3 Siamese Networks and Triplet Loss

A Siamese network is a neural architecture in which the same encoder weights are applied to two or more inputs in parallel, and the network is trained on a pairwise or triplet objective that depends on the distances between the resulting embeddings. The name comes from the symmetry of the architecture (a single shared encoder, applied multiple times). The training objective is geometric: the encoder should map inputs that are semantically similar to nearby points in the embedding space, and inputs that are semantically different to distant points.

The triplet loss [9] formalises this. For an anchor input x_a , a positive input x_+ from the same class, and a negative input x_- from a different class, the triplet loss is:

$$L_{triplet} = \max(0, \|f(x_a) - f(x_+)\|^2 - \|f(x_a) - f(x_-)\|^2 + \alpha) \quad [\text{Eq. B1}]$$

where $f(\cdot)$ is the encoder and $\alpha > 0$ is a margin parameter (typically $\alpha = 0.5$ on normalised embeddings). The loss is zero when the positive is at least α closer to the anchor than the negative is; otherwise, it produces a gradient that pulls the positive closer and pushes the negative further away. In the FusionCore v_1 advanced ablation, the anchor is a 30-cycle window from one engine, the positive is a window from a different engine in the same risk band, and the negative is a window from a different risk band. The triplet loss thereby shapes the embedding space along the health-state axis.

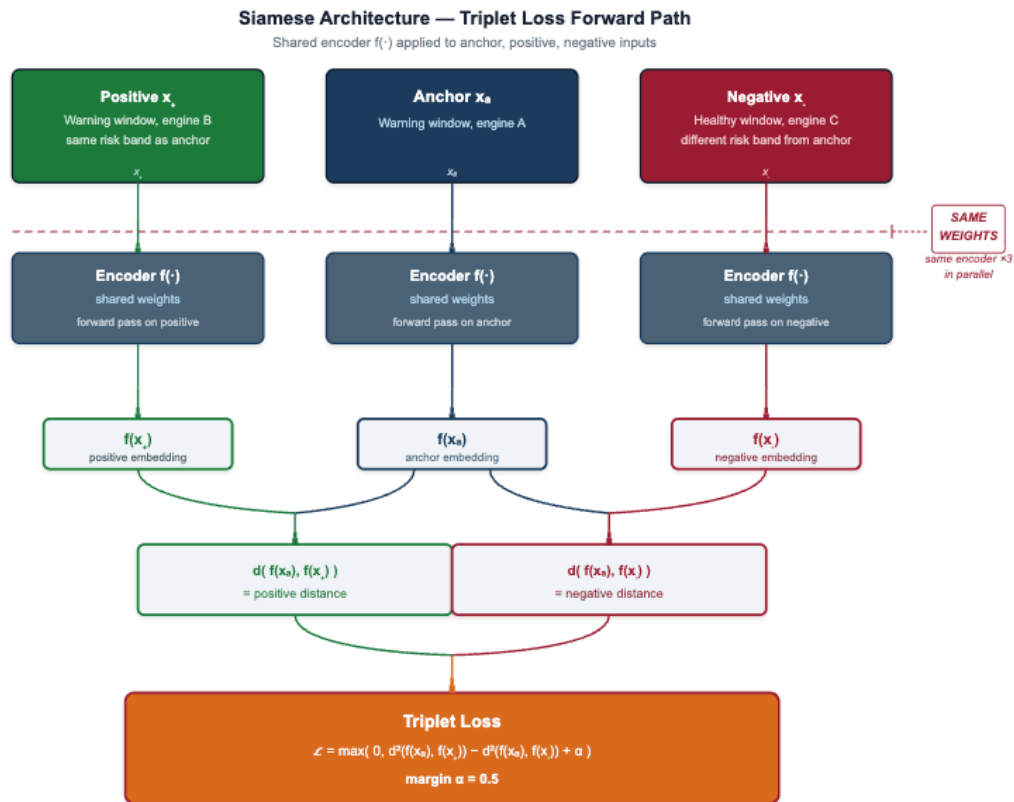


Figure 3: Siamese architecture with triplet loss: shared encoder $f(\cdot)$ applied in parallel to anchor, positive, and negative inputs; loss enforces margin $\alpha = 0.5$ between positive and negative pair distances.

A practical refinement of the basic triplet loss is "semi-hard triplet mining," introduced by Schroff et al. [9]. The motivation is that not all triplets are equally informative for learning. Easy triplets — where the negative is already far from the anchor and the loss is zero — provide no gradient and waste computation. Hard triplets — where the negative is closer to the anchor than the positive is — produce large, conflicting gradients that can destabilise training. The informative middle band is the "semi-hard" region: triplets where the negative is further from the anchor than the positive is (so the basic ordering constraint is already satisfied) but by less than the margin α . These triplets generate a small, stable gradient that pulls the negative outside the margin without overshooting. The mining procedure happens once per batch: the encoder forward-passes all anchors, positives, and negatives; pairwise distances are computed; and only triplets whose negative falls in the semi-hard zone are retained for the loss computation. A worked example is given in §A.10.

A.4 t-SNE and Kolmogorov–Smirnov Embedding Diagnostics

t-SNE (t-distributed Stochastic Neighbour Embedding) [19] is a dimensionality-reduction technique that produces a 2D or 3D scatter visualisation of high-dimensional data while attempting to preserve local neighbourhood structure. In the advanced ablation it is used as a visual diagnostic of the trained PiNet embedding: if the metric-learning regularisation is working as intended, points coloured by risk band should form visibly separated clusters in the t-SNE projection. t-SNE is exclusively a visual diagnostic — its output is not used for any quantitative gate decision.

The Kolmogorov–Smirnov (KS) test is the corresponding quantitative gate. It is a non-parametric test of whether two empirical distributions differ significantly. For embedding evaluation, it is applied to two distributions of pairwise Euclidean distances: the same-stage distance distribution (distances between embedding pairs from the same risk band) and the different-stage distribution (distances between pairs from different risk bands). A well-structured embedding should have substantially smaller distances within-class than between-class. The KS test computes the maximum absolute difference between the two empirical cumulative distribution functions and reports a p -value. A p -value below 0.05 with a same-stage mean less than 0.67 of the different-stage mean is the embedding-quality gate for the advanced ablation. A failed gate is reported as a null result; it does not prevent the rest of v_1 from completing.

A.5 Cumulative Fatigue Index (CFI)

The Cumulative Fatigue Index family is the FusionCore proxy for Miner's Rule cumulative damage [18]. Miner's Rule states that fatigue damage accumulates linearly in the cycle ratio n_k/N_k for each stress level k , with failure at total accumulation equal to unity. In closed-form gas-turbine fatigue analysis, computing n_k and N_k requires destructive testing data that is not available in the C-MAPSS simulator; the FusionCore proxy substitutes running accumulation of positive sensor deviations from the within-regime healthy baseline as a regime-normalised approximation of cumulative stress above the healthy operating state.

For sensor s at cycle t , the proxy CFI is computed cumulatively from cycle 0 of each engine trajectory:

$$CFI_s(t) = \sum_{\tau=0}^t \max(0, z_{s,\tau} - z_s^{\text{healthy}}) \quad [\text{Eq. B2}]$$

where $z_{s,\tau}$ is the regime-normalised z -score of sensor s at cycle τ and z_s^{healthy} is the per-regime healthy baseline. By construction the index is non-decreasing within an engine trajectory — Phase 3 Gate 6 of v_0 confirmed this empirically on the full FD00u corpus.

A.6 Cox Proportional Hazards Model

The Cox proportional hazards model [17] is a semi-parametric regression model for the hazard function — the instantaneous probability of failure conditional on having survived to time t — under the proportional hazards' assumption that hazard ratios between subjects are constant in time:

$$h(t|X) = h_0(t) \exp(\beta^T X) \quad [\text{Eq. B3}]$$

The baseline hazard $h_0(t)$ cancels from the partial likelihood used to estimate β , allowing the regression coefficients to be estimated without specifying the failure-time distribution. In FusionCore, the partial hazard score $\exp(\hat{\beta}^T X_t)$ is computed per cycle using the $\hat{\beta}$ frozen from v_0 Phase 3 G12 and is included as a physics token in the PiNet physics branch. This operationalises the survival-analysis information directly in the PiNet input space rather than as a separate auxiliary output head.

A.7 What is a "motif"?

In time-series machine learning, a motif is a short subsequence pattern that recurs across a longer signal. The term is borrowed from molecular biology (where a motif is a recurring DNA pattern) and from music (where a motif is a recurring melodic phrase). For turbofan degradation, motifs would be patterns such as a sudden 2–3 cycle dip in compressor pressure ratio followed by a slow recovery (a "wash event"); a 5-cycle volatility burst in core speed (a transient bearing anomaly); a step jump in EGT followed by a new elevated baseline (a degradation onset). A convolutional neural network is structurally well-suited to learning these short patterns because its kernels slide over time and detect local shapes regardless of where in the trajectory they occur. The reason PiNet does not include a convolutional branch is that the FusionCore v_0 feature engineering already extracted the same information into explicit kinematic features (5-cycle and 10-cycle rolling means and standard deviations), so a convolutional pathway would learn what is already in its input.

A.8 Worked Example — Why $\lambda_{class} = 0.3$

Consider a training batch of 64 windows drawn from the FusionCore v_0 feature manifold under the recommended batch composition: approximately 28 Healthy, 18 Warning, 18 Critical (Table 10). Suppose for illustrative purposes, that the mean per-example loss values mid-training are: $L_{RUL} = 8.5$ cycles² (a moderate squared-error on RUL prediction); $L_{CE,Healthy} = 0.4$ nats (well-calibrated); $L_{CE,Warning} = 1.1$ nats (moderate confusion with Healthy); $L_{CE,Critical} = 2.3$ nats (poorly calibrated owing to under-representation). The class-weighted CE applies inverse-frequency weights of approximately {0.55, 1.24, 2.53} to Healthy, Warning, Critical, producing a weighted batch-mean $L_{CE} \approx 2.8$ nats.

Under the loss in Eq. 3, the relative contribution of the classification term to the total loss is $\lambda_{class} \times L_{CE}/L_{RUL} = \lambda_{class} \times 2.8/8.5 = 0.33\lambda_{class}$. At $\lambda_{class} = 1.0$ the classification term contributes 33% of the total loss; at $\lambda_{class} = 0.3$ it contributes 10%. The gradient with respect to the shared embedding e flows from both heads in proportion to these contributions. A 10% classification gradient share is sufficient to keep the classification head learning without destabilising the regression objective and is consistent with the operational priority sequence agreed in §5: NASA score and Critical recall are decisive metrics, but RMSE on the regression head is the headline number against the XGBoost benchmark.

A.9 Worked Example — Class-Weighted Cross-Entropy

Standard (unweighted) cross-entropy treats every training example as equally important. In an imbalanced dataset, this produces a model that is well-calibrated for the majority class and poorly calibrated for the minority class, because the majority class dominates the gradient signal by sheer numerical weight. The simplest mitigation, used by FusionCore v_1 , is to multiply the per-example cross-entropy loss by a weight that is inversely proportional to the class frequency.

A worked example clarifies this. The Internal Train partition contains approximately 129,331 rows. Of these, the class breakdown is: Healthy ($RUL \geq 80$) approximately 78,000 rows (60%); Warning ($30 \leq RUL < 80$) approximately 34,000 rows (26.3%); Critical ($RUL < 30$) approximately 17,000 rows (13.2%). The 9:1 structural imbalance referenced in §2.3 is the ratio between the most under-represented terminal-band rows within an individual long trajectory (a 300-cycle engine has only ~30

Critical-band cycles against ~270 non-Critical) — note that this is a within-trajectory ratio, distinct from the dataset-level Healthy:Critical ratio of approximately 4.5:1.

The inverse-frequency weight for class c is computed once on Internal Train and frozen as $w_c = N/(3 \times n_c)$, where N is the total row count and n_c is the count for class c . Numerically: $w_{Healthy} = 129331/(3 \times 78000) \approx 0.55$; $w_{Warning} = 129331/(3 \times 34000) \approx 1.27$; $w_{Critical} = 129331/(3 \times 17000) \approx 2.54$. The class-weighted cross-entropy on a single training example i in class c_i becomes $L_{CE,i} = w_{c_i} \times CE(\hat{p}_i, c_i)$. The effect is that a single misclassified Critical example contributes $2.54/0.55 \approx 4.6$ times more to the gradient than a single misclassified Healthy example, which restores the gradient-level balance between the three classes despite the underlying frequency imbalance.

A.10 Worked Example — Semi-Hard Triplet Mining

Suppose a single training batch contains an anchor window x_a (a Warning-band window from engine FD002_47), a positive window x_+ (a Warning-band window from a different engine, FD004_119), and three candidate negatives drawn from Healthy-band windows. After the encoder forward pass, the pairwise embedding distances are: $d(x_a, x_+) = 0.62$; candidate negatives at distances $d_1 = 1.85$, $d_2 = 0.95$, $d_3 = 0.40$. With margin $\alpha = 0.5$, the boundary for "semi-hard" is the interval $[d(x_a, x_+), d(x_a, x_+) + \alpha] = [0.62, 1.12]$.

The candidate at $d_1 = 1.85$ is "easy" — it is already well outside the margin and contributes zero gradient. The candidate at $d_3 = 0.40$ is "hard" — it is closer to the anchor than the positive is, which means the embedding has its sign wrong; using this triplet would generate a large gradient that risks destabilising training. The candidate at $d_2 = 0.95$ is "semi-hard" — it sits inside the margin band, so the model knows the negative ordering is correct, but the margin is not yet satisfied; this triplet generates a small, well-conditioned gradient that pulls d_2 outside the margin band over the next few epochs. Semi-hard mining selects only this third candidate for the triplet loss computation in this batch. Across thousands of batches, this selection rule produces the gradual, stable margin enforcement that distinguishes triplet metric learning from contrastive or Siamese losses.

A.11 Entropy Depletion and the Shannon Regulariser

In information theory, the Shannon entropy of a probability distribution measures how spread out — how uncertain — the distribution is. A distribution like (0.33,0.33,0.33) across three classes is maximally uncertain (entropy ≈ 1.58 nats). A distribution like (1.0,0.0,0.0) is maximally certain (entropy = 0 nats). When a classifier becomes over-confident — typically because it has been over-trained on a metric-learning objective without sufficient regularisation — its predicted distributions collapse toward the certain end of this scale. This is "entropy depletion".

In the context of triplet metric learning, entropy depletion manifests as embedding collapse. The encoder learns to map all examples of a given class to a single, very tight cluster in embedding space. The triplet loss is satisfied by construction (any positive is closer than any negative to any anchor), but the model has lost all information about within-class variation. For PiNet, this would mean that two engines both classified as Warning would produce identical embedding outputs, even if one is at $RUL = 75$ and the other at $RUL = 35$. The downstream RUL regression head would then have nothing useful to regress against and would degenerate to predicting the within-class mean.

The Shannon entropy regulariser counteracts this by adding a term to the loss that rewards higher entropy — that is, more spread-out probability distributions. In its simplest form, the regulariser is the negative entropy of the per-class soft assignments averaged across the batch: minimising the loss thereby maximises entropy. The combination with triplet loss is now well-motivated: triplet loss enforces between-class separation, Shannon regularisation preserves within-class variation, and together they produce an embedding space that is both discriminable and informative.

A.12 Forward-Fill Padding for Short Trajectories

The TCN consumes fixed-length 30-cycle windows. For a trajectory of length ≥ 30 cycles (the vast majority of C-MAPSS engines: minimum trajectory length in FD001 is 128 cycles, and the corresponding minima in FD002, FD003, and FD004 are even longer), this is straightforward — windows are constructed by sliding from cycle 0 to cycle $T - W$, where T is the engine's terminal cycle. For the early cycles of any trajectory (cycles 0 to 29), however, the standard sliding window cannot be constructed: the engine has not yet accumulated 30 cycles of history.

Three padding strategies are commonly used in time-series modelling: zero-padding (replace missing cycles with zero vectors), forward-fill (replicate the first observed cycle leftward), and trajectory truncation (skip the first 30 cycles entirely). FusionCore v_1 uses forward-fill. The reason zero-padding is rejected is that, after z -score normalisation against the regime baseline, $z = 0$ corresponds to the average cycle in that regime — meaning a zero vector represents an *average* engine, not a *missing* engine. This would inject misleading "average healthy" signal into early training windows. Forward-fill replicates the first observed cycle, which after normalisation is still close to but not exactly $z = 0$ (because the first observed cycle was sampled, not synthesised), and is therefore a less misleading approximation of pre-trajectory behaviour.

In practice, an attention mask attached to each window flags the padded positions. The TCN forward pass uses this mask to zero out the contribution of padded positions to the loss gradient, so the model learns from real cycles only. Trajectory truncation was rejected because it would discard the early cycles entirely from training, potentially losing healthy-band examples in subsets where they are scarce.